

[VN] FastAPI hoạt động dựa trên ASGI như thế nào?

[EN] How does FastAPI work based on ASGI?

[VN] FastAPI hoạt động dựa trên ASGI (Asynchronous Server Gateway Interface), là chuẩn giao tiếp giữa web server và ứng dụng Python hỗ trợ bất đồng bộ. Khi có request đến, server ASGI như Uvicorn sẽ nhận request và chuyển vào ứng dụng FastAPI dưới dạng event loop. FastAPI sử dụng `async/await` để xử lý nhiều request đồng thời mà không block thread, đặc biệt hiệu quả với I/O như gọi database hoặc API bên ngoài. Bên dưới, FastAPI dựa trên Starlette để xử lý routing và middleware, và dùng Pydantic để validate dữ liệu. Nhờ ASGI, FastAPI có thể xử lý concurrency tốt hơn so với WSGI truyền thống.

[EN] FastAPI is built on ASGI (Asynchronous Server Gateway Interface), which is a standard interface between a web server and a Python application that supports asynchronous processing. When a request comes in, an ASGI server like Uvicorn receives it and passes it to the FastAPI application through an event loop. FastAPI uses `async/await` to handle multiple requests concurrently without blocking threads, which is very efficient for I/O operations such as database queries or external API calls. Internally, FastAPI is based on Starlette for routing and middleware, and uses Pydantic for data validation. Thanks to ASGI, FastAPI provides better concurrency compared to traditional WSGI applications.

[VN] `async/await` trong Python hoạt động ra sao?

[EN] How do `async/await` work in Python?

[VN] Trong Python, `async/await` được dùng để viết code bất đồng bộ dựa trên event loop. Khi một hàm được khai báo với `async`, nó trở thành coroutine và có thể tạm dừng tại điểm `await` mà không block toàn bộ chương trình. Khi gặp `await` (ví dụ chờ gọi database hoặc API), coroutine sẽ nhường quyền cho event loop để xử lý các task khác trong lúc chờ I/O hoàn thành. Khi kết quả sẵn sàng, event loop sẽ tiếp tục chạy phần còn lại của coroutine. Cơ chế này giúp xử lý nhiều request đồng thời hiệu quả hơn, đặc biệt với các tác vụ I/O-bound, nhưng không làm tăng tốc các tác vụ CPU-bound.

[EN] In Python, `async/await` is used to write asynchronous code based on an event loop. When a function is defined with `async`, it becomes a coroutine and can pause execution at an `await` statement without blocking the entire program. When the code reaches `await` (for example waiting for a database query or external API call), the coroutine yields control back to the event loop so other tasks can run while waiting for I/O to complete. Once the result is ready, the event loop resumes the coroutine execution. This mechanism allows efficient handling of multiple concurrent requests, especially for I/O-bound tasks, but it does not speed up CPU-bound operations.

[VN] Khi nào nên dùng async? Khi nào không?

[EN] When should async be used? When should it not be used?

[VN] Nên dùng async khi ứng dụng có nhiều tác vụ I/O-bound như gọi database, gọi API bên ngoài, đọc/ghi file hoặc xử lý nhiều request web đồng thời. Trong các trường hợp này, async giúp không block thread trong lúc chờ I/O và tăng khả năng xử lý concurrent request, đặc biệt trong web framework như FastAPI. Tuy nhiên, không nên dùng async cho tác vụ CPU-bound như xử lý tính toán nặng, mã hóa, xử lý hình ảnh phức tạp vì async không làm tăng tốc CPU. Trong trường hợp đó nên dùng multiprocessing hoặc worker riêng. Ngoài ra, nếu hệ thống nhỏ, ít concurrent request hoặc team chưa quen async, dùng code đồng bộ có thể đơn giản và dễ maintain hơn.

[EN] You should use async when the application has many I/O-bound tasks such as database calls, external API requests, file operations, or handling many concurrent web requests. In these cases, async prevents blocking while waiting for I/O and improves concurrency, especially in web frameworks like FastAPI. However, you should not use async for CPU-bound tasks such as heavy computation, encryption, or complex image processing because async does not improve CPU performance. In those cases, multiprocessing or separate workers are better solutions. Also, if the system is small, has low concurrency, or the team is not familiar with async, synchronous code may be simpler and easier to maintain.

[VN] So sánh FastAPI và Flask về performance và concurrency.

[EN] Compare FastAPI and Flask in terms of performance and concurrency.

[VN] FastAPI và Flask đều là web framework của Python nhưng khác nhau về performance và concurrency. FastAPI được xây dựng trên ASGI và hỗ trợ async/await, nên có thể xử lý nhiều request đồng thời hiệu quả hơn, đặc biệt với các tác vụ I/O-bound như gọi database hoặc API bên ngoài. Nhờ đó, FastAPI thường có throughput cao hơn và latency thấp hơn trong môi trường concurrent cao. Flask được xây dựng trên WSGI và mặc định là synchronous, nên mỗi request có thể block thread cho đến khi hoàn thành. Flask vẫn có thể scale bằng cách tăng số worker (ví dụ dùng Gunicorn), nhưng concurrency không linh hoạt bằng FastAPI. Tuy nhiên, Flask đơn giản, nhẹ và phù hợp cho ứng dụng nhỏ hoặc không cần xử lý async phức tạp.

[EN] FastAPI and Flask are both Python web frameworks but differ in performance and concurrency. FastAPI is built on ASGI and supports async/await, so it can handle many concurrent requests more efficiently, especially for I/O-bound tasks like database or external API calls. Because of this, FastAPI usually provides higher throughput and lower latency under high concurrency. Flask is built on WSGI and is synchronous by default, meaning each request can block a worker until it finishes. Flask can still scale by

increasing the number of workers (for example using Gunicorn), but its concurrency model is less flexible than FastAPI. However, Flask is simple, lightweight, and suitable for small applications or systems that do not require complex async handling.

[VN] Pydantic validation hoạt động như thế nào?

[EN] How does Pydantic validation work?

[VN] Pydantic validation hoạt động bằng cách định nghĩa các model dựa trên type hint của Python. Khi một request được gửi đến FastAPI, dữ liệu JSON sẽ được parse vào Pydantic model. Pydantic sẽ tự động kiểm tra kiểu dữ liệu, giá trị bắt buộc, giới hạn độ dài hoặc điều kiện tùy chỉnh. Nếu dữ liệu không hợp lệ, nó sẽ trả về lỗi chi tiết mà không cần viết code validate thủ công. Pydantic cũng hỗ trợ chuyển đổi kiểu dữ liệu, ví dụ từ string sang datetime hoặc int nếu có thể. Cơ chế này giúp đảm bảo dữ liệu đầu vào luôn đúng format và giảm lỗi trong business logic.

[EN] Pydantic validation works by defining models based on Python type hints. When a request is sent to FastAPI, the JSON data is parsed into a Pydantic model. Pydantic automatically validates data types, required fields, length limits, and custom conditions. If the data is invalid, it returns detailed validation errors without needing manual validation code. Pydantic can also convert data types automatically, for example from string to datetime or int when possible. This mechanism ensures that input data is in the correct format and reduces errors in business logic.

[VN] Middleware trong FastAPI dùng để làm gì?

[EN] What is middleware used for in FastAPI?

[VN] Middleware trong FastAPI được dùng để xử lý request và response trước hoặc sau khi vào endpoint chính. Nó hoạt động như một lớp trung gian giữa client và application. Middleware thường được dùng để logging, kiểm tra authentication, thêm header, xử lý CORS, đo thời gian xử lý request hoặc xử lý lỗi chung. Khi một request đến, middleware có thể thực hiện logic trước, sau đó gọi tiếp đến endpoint, và sau khi endpoint trả về response, middleware có thể chỉnh sửa hoặc thêm thông tin trước khi gửi lại cho client. Cách này giúp tách biệt các chức năng chung khỏi business logic và giữ code sạch hơn.

[EN] Middleware in FastAPI is used to process requests and responses before or after they reach the main endpoint. It works as an intermediate layer between the client and the application. Middleware is commonly used for logging, authentication checks, adding headers, handling CORS, measuring request processing time, or managing global errors. When a request comes in, the middleware can execute logic first, then pass control to the endpoint, and after the endpoint returns a response, the middleware can modify or add

information before sending it back to the client. This approach separates common concerns from business logic and keeps the code cleaner.

[VN] Làm sao tối ưu performance API Python?

[EN] How to optimize Python API performance?

[VN] Để tối ưu performance API Python, tôi bắt đầu bằng việc xác định bottleneck thông qua profiling và monitoring. Với I/O-bound workload, tôi dùng async/await (ví dụ FastAPI) để tăng concurrency và giảm blocking. Tôi tối ưu database bằng cách tạo index phù hợp, tránh N+1 query, dùng connection pool và cache dữ liệu thường xuyên truy cập bằng Redis. Tôi cũng giảm overhead bằng cách dùng JSON serialization nhanh, bật gzip nếu cần và tối ưu logic xử lý trong code. Ở mức hệ thống, tôi deploy phía sau load balancer, scale ngang bằng nhiều worker hoặc container và cấu hình resource hợp lý. Ngoài ra, tôi sử dụng logging và tracing để theo dõi latency và cải thiện liên tục.

[EN] To optimize Python API performance, I first identify bottlenecks using profiling and monitoring tools. For I/O-bound workloads, I use async/await (for example with FastAPI) to increase concurrency and reduce blocking. I optimize the database by adding proper indexes, avoiding N+1 queries, using connection pooling, and caching frequently accessed data with Redis. I also reduce overhead by using efficient JSON serialization, enabling gzip if needed, and improving business logic efficiency. At the system level, I deploy behind a load balancer, scale horizontally with multiple workers or containers, and configure proper resource limits. Additionally, I use logging and tracing to monitor latency and continuously improve performance.

[VN] GIL là gì? Nó ảnh hưởng thế nào tới multi-threading?

[EN] What is the GIL and how does it affect multi-threading?

[VN] GIL (Global Interpreter Lock) là một cơ chế trong CPython cho phép chỉ một thread thực thi bytecode Python tại một thời điểm. Điều này có nghĩa là dù tạo nhiều thread, chỉ một thread có thể chạy code Python cùng lúc. Vì vậy, với tác vụ CPU-bound, multi-threading trong Python không giúp tăng hiệu năng do các thread phải chia sẻ GIL. Tuy nhiên, với tác vụ I/O-bound như gọi API hoặc truy vấn database, thread có thể nhả GIL khi chờ I/O, nên multi-threading vẫn cải thiện được concurrency. Nếu cần xử lý CPU nặng, nên dùng multiprocessing hoặc các giải pháp khác để tận dụng nhiều core CPU.

[EN] GIL (Global Interpreter Lock) is a mechanism in CPython that allows only one thread to execute Python bytecode at a time. This means that even if multiple threads are created, only one thread can run Python code simultaneously. As a result, for CPU-bound tasks, multi-threading in Python does not improve performance because threads must share the GIL. However, for I/O-bound tasks such as API calls or database queries, a thread can

release the GIL while waiting for I/O, so multi-threading can still improve concurrency. For heavy CPU processing, it is better to use multiprocessing or other solutions to utilize multiple CPU cores.

[VN] Multiprocessing vs Threading trong Python.

[EN] Multiprocessing vs Threading in Python.

[VN] Trong Python, threading tạo nhiều thread trong cùng một process và chia sẻ chung bộ nhớ. Thread nhẹ, tạo nhanh và phù hợp với tác vụ I/O-bound như gọi API, đọc file hoặc truy vấn database. Tuy nhiên, do GIL, threading không cải thiện hiệu năng cho tác vụ CPU-bound vì chỉ một thread chạy Python bytecode tại một thời điểm. Ngược lại, multiprocessing tạo nhiều process riêng biệt, mỗi process có bộ nhớ và GIL riêng, nên có thể tận dụng nhiều core CPU để xử lý tác vụ CPU-bound hiệu quả hơn. Nhược điểm là multiprocessing tốn tài nguyên hơn và việc giao tiếp giữa các process phức tạp hơn so với thread.

[EN] In Python, threading creates multiple threads within the same process and shares the same memory space. Threads are lightweight, faster to create, and suitable for I/O-bound tasks such as API calls, file operations, or database queries. However, because of the GIL, threading does not improve performance for CPU-bound tasks since only one thread can execute Python bytecode at a time. In contrast, multiprocessing creates separate processes, each with its own memory space and its own GIL, allowing true parallel execution on multiple CPU cores for CPU-bound tasks. The downside is that multiprocessing uses more system resources and inter-process communication is more complex compared to threading.

[VN] Cách implement background tasks trong FastAPI.

[EN] How to implement background tasks in FastAPI.

[VN] Trong FastAPI, có thể implement background tasks bằng cách sử dụng BackgroundTasks được cung cấp sẵn. Khi endpoint nhận request, ta thêm task vào BackgroundTasks, và sau khi response được trả về cho client, FastAPI sẽ thực thi task đó ở phía sau. Cách này phù hợp cho các tác vụ nhẹ như gửi email, ghi log hoặc gọi webhook mà không cần chờ hoàn thành trước khi trả response. Với các tác vụ nặng hoặc cần xử lý lâu dài, nên dùng message queue như SQS hoặc Kafka và worker riêng để xử lý bất đồng bộ, giúp hệ thống ổn định và scale tốt hơn.

[EN] In FastAPI, background tasks can be implemented using the built-in BackgroundTasks feature. When an endpoint receives a request, we add a task to BackgroundTasks, and after the response is sent to the client, FastAPI executes the task in the background. This approach is suitable for lightweight tasks such as sending emails,

writing logs, or calling webhooks without delaying the response. For heavy or long-running tasks, it is better to use a message queue like SQS or Kafka with separate workers to process asynchronously, which improves system stability and scalability.

[VN] Rate limiting trong API nên làm ở đâu?

[EN] Where should rate limiting be implemented in an API?

[VN] Rate limiting trong API nên được đặt ở nhiều lớp để đảm bảo bảo vệ hệ thống hiệu quả. Tốt nhất nên thực hiện ở API Gateway hoặc Load Balancer vì đây là điểm vào chung, giúp chặn request quá nhiều trước khi vào service bên trong, giảm tải hệ thống. Ngoài ra, có thể implement thêm rate limiting ở mức application nếu cần kiểm soát chi tiết theo user, API key hoặc endpoint cụ thể. Thông thường, rate limit sẽ dựa trên IP, user ID hoặc token và lưu counter trong Redis để đảm bảo hiệu năng cao. Cách thiết kế nhiều lớp giúp tăng bảo mật, tránh abuse và bảo vệ tài nguyên backend.

[EN] Rate limiting in an API should be implemented at multiple layers for better protection. Ideally, it should be placed at the API Gateway or Load Balancer level because this is the common entry point, allowing excessive requests to be blocked before reaching internal services, reducing system load. Additionally, rate limiting can be implemented at the application level when more fine-grained control is needed per user, API key, or specific endpoint. Rate limits are usually based on IP address, user ID, or token, and counters are stored in Redis for high performance. A multi-layer design improves security, prevents abuse, and protects backend resources.

[VN] Làm sao handle global exception trong FastAPI?

[EN] How to handle global exceptions in FastAPI?

[VN] Trong FastAPI, để handle global exception, tôi sử dụng `@app.exception_handler` để định nghĩa một handler chung cho từng loại exception hoặc cho Exception tổng quát. Khi có lỗi xảy ra trong bất kỳ endpoint nào, FastAPI sẽ chuyển lỗi đó vào handler tương ứng và trả về response chuẩn hóa, ví dụ với message rõ ràng và HTTP status code phù hợp. Tôi thường tạo một format response thống nhất cho error như gồm `error_code`, message và details để client dễ xử lý. Ngoài ra, có thể dùng middleware để log toàn bộ lỗi trước khi trả response. Cách này giúp hệ thống dễ maintain, dễ debug và đảm bảo API trả lỗi nhất quán.

[EN] In FastAPI, to handle global exceptions, I use `@app.exception_handler` to define a common handler for specific exception types or for the general Exception class. When an error occurs in any endpoint, FastAPI routes it to the corresponding handler and returns a standardized response, for example with a clear message and proper HTTP status code. I usually define a consistent error response format including fields like `error_code`, message, and details so clients can handle errors easily. Additionally, middleware can be used to log

all exceptions before returning the response. This approach makes the system easier to maintain, easier to debug, and ensures consistent error handling across the API.

[VN] Làm sao viết API versioning?

[EN] How to implement API versioning?

[VN] Để viết API versioning, tôi thường áp dụng version trong URL như /api/v1/... và /api/v2/... vì cách này rõ ràng, dễ quản lý và dễ maintain. Khi có thay đổi lớn hoặc breaking change, tôi tạo version mới thay vì sửa version cũ, và vẫn giữ version cũ hoạt động trong một thời gian để client có thể migrate dần. Ngoài version trong URL, cũng có thể dùng version trong header (ví dụ Accept-Version) nhưng cách này ít phổ biến hơn. Tôi đảm bảo mỗi version có contract rõ ràng bằng OpenAPI hoặc protobuf và có tài liệu đầy đủ. Cách này giúp hệ thống phát triển linh hoạt mà không làm ảnh hưởng client hiện tại.

[EN] To implement API versioning, I usually include the version in the URL such as /api/v1/... and /api/v2/... because it is clear, easy to manage, and easy to maintain. When there is a major or breaking change, I create a new version instead of modifying the old one, and keep the old version running for a period of time so clients can migrate gradually. Besides URL versioning, versioning can also be done using headers (for example Accept-Version), but this is less common. I ensure each version has a clear contract using OpenAPI or protobuf and proper documentation. This approach allows the system to evolve without breaking existing clients.

[VN] Bạn sẽ test API như thế nào (unit/integration)?

[EN] How would you test an API (unit/integration)?

[VN] Để test API, tôi chia thành hai mức: unit test và integration test. Với unit test, tôi test từng function hoặc từng endpoint logic riêng lẻ, thường dùng pytest và mock các dependency như database hoặc service bên ngoài để đảm bảo test nhanh và cô lập. Mục tiêu là kiểm tra business logic hoạt động đúng với nhiều trường hợp khác nhau, bao gồm cả case thành công và lỗi. Với integration test, tôi test luồng hoàn chỉnh từ request đến database hoặc service thật (có thể dùng test database), để đảm bảo các thành phần hoạt động đúng khi kết hợp với nhau. Tôi cũng kiểm tra HTTP status code, response format và validation. Cách này giúp phát hiện lỗi sớm ở mức nhỏ và đảm bảo toàn bộ hệ thống hoạt động đúng trước khi deploy.

[EN] To test an API, I divide testing into two levels: unit test and integration test. For unit tests, I test individual functions or endpoint logic separately, usually using pytest and mocking dependencies such as the database or external services to keep tests fast and isolated. The goal is to verify that the business logic works correctly in different scenarios, including both success and error cases. For integration tests, I test the full flow from

request to database or real services (often using a test database) to ensure all components work correctly together. I also verify HTTP status codes, response format, and validation behavior. This approach helps detect issues early at a small level and ensures the whole system works properly before deployment.

[VN] Python memory management hoạt động thế nào?

[EN] How does Python memory management work?

[VN] Python quản lý bộ nhớ chủ yếu bằng reference counting, nghĩa là mỗi object sẽ có một bộ đếm số lượng biến đang tham chiếu đến nó; khi bộ đếm về 0, object sẽ được giải phóng bộ nhớ. Ngoài ra, Python còn có garbage collector để xử lý circular reference (các object tham chiếu lẫn nhau mà reference count không về 0). Bộ nhớ được cấp phát qua một bộ quản lý riêng (Python memory allocator) để tối ưu hiệu năng. Nhờ cơ chế này, lập trình viên ít khi phải giải phóng bộ nhớ thủ công, nhưng vẫn có thể gặp memory leak nếu giữ tham chiếu không cần thiết hoặc tạo vòng tham chiếu phức tạp.

[EN] Python manages memory mainly using reference counting, meaning each object keeps a count of how many variables reference it; when the count reaches zero, the object is freed. In addition, Python has a garbage collector to handle circular references, where objects reference each other and the count does not drop to zero. Memory is allocated through Python's own memory allocator to improve performance. Because of this system, developers usually do not need to free memory manually, but memory leaks can still happen if unnecessary references are kept or complex circular references are created.

[VN] Reference counting và garbage collector khác nhau ra sao?

[EN] How do reference counting and garbage collector differ?

[VN] Reference counting là cơ chế quản lý bộ nhớ trong Python, trong đó mỗi object giữ một bộ đếm số lượng tham chiếu; khi số tham chiếu giảm về 0 thì object được giải phóng ngay lập tức. Garbage collector (GC) là cơ chế bổ sung để xử lý các circular reference, tức là các object tham chiếu lẫn nhau làm cho reference count không về 0. Reference counting hoạt động liên tục và thu hồi bộ nhớ nhanh, còn garbage collector chạy định kỳ để quét và dọn dẹp các vòng tham chiếu mà reference counting không xử lý được.

[EN] Reference counting is a memory management mechanism in Python where each object keeps a count of how many references point to it; when the count reaches zero, the object is immediately freed. The garbage collector (GC) is an additional mechanism that handles circular references, where objects reference each other and the reference count does not drop to zero. Reference counting works continuously and frees memory quickly, while the garbage collector runs periodically to detect and clean up cycles that reference counting cannot resolve.

[VN] Mutable vs immutable ảnh hưởng thế nào đến performance và bug?

[EN] How do mutable vs immutable affect performance and bugs?

[VN] Trong Python, mutable object (như list, dict) có thể thay đổi giá trị sau khi tạo, còn immutable object (như int, str, tuple) thì không thể thay đổi mà sẽ tạo object mới khi chỉnh sửa. Về performance, mutable thường tiết kiệm bộ nhớ hơn khi thay đổi nhiều lần vì không cần tạo object mới, nhưng có thể gây bug do bị thay đổi ngoài ý muốn khi truyền vào hàm hoặc dùng chung reference. Immutable an toàn hơn vì không bị thay đổi ngầm, giúp tránh bug liên quan đến shared state, nhưng có thể tốn thêm bộ nhớ khi tạo object mới nhiều lần. Vì vậy, immutable giúp code an toàn hơn, còn mutable linh hoạt hơn nhưng cần cẩn thận.

[EN] In Python, mutable objects (such as list, dict) can change their value after creation, while immutable objects (such as int, str, tuple) cannot be changed and will create a new object when modified. In terms of performance, mutable objects can save memory when updated many times because they do not need to create new objects, but they may cause bugs due to unexpected changes when passed to functions or shared across references. Immutable objects are safer because they cannot be changed silently, reducing bugs related to shared state, but they may use more memory when new objects are created frequently. Therefore, immutable improves safety, while mutable provides flexibility but requires caution.

[VN] Deep copy vs shallow copy khác nhau thế nào?

[EN] How do deep copy and shallow copy differ?

[VN] Shallow copy tạo một object mới nhưng các phần tử bên trong vẫn tham chiếu đến object gốc, nên nếu thay đổi dữ liệu lồng bên trong (như list trong list) thì cả bản sao và bản gốc đều bị ảnh hưởng. Deep copy tạo một bản sao hoàn toàn mới, bao gồm cả các object con bên trong, nên thay đổi ở bản sao sẽ không ảnh hưởng đến bản gốc. Shallow copy nhanh và tốn ít bộ nhớ hơn, nhưng dễ gây bug khi làm việc với cấu trúc dữ liệu lồng nhau; deep copy an toàn hơn nhưng tốn tài nguyên hơn.

[EN] Shallow copy creates a new object, but its inner elements still reference the original objects, so if you change nested data (like a list inside a list), both the copy and the original are affected. Deep copy creates a completely new object, including all nested objects, so changes in the copy do not affect the original. Shallow copy is faster and uses less memory, but it can cause bugs when working with nested data structures; deep copy is safer but uses more resources.

[VN] Khi nào gây bug khó phát hiện?

[EN] When do hard-to-detect bugs occur?

[VN] Bug khó phát hiện thường xảy ra khi dùng mutable object chung giữa nhiều biến hoặc nhiều hàm, đặc biệt là khi dùng shallow copy với dữ liệu lồng nhau, vì thay đổi ở một nơi sẽ ảnh hưởng nơi khác mà không rõ ràng. Nó cũng xảy ra khi có shared state trong multi-threading hoặc async mà không kiểm soát đồng bộ. Ngoài ra, default mutable argument trong hàm (ví dụ list hoặc dict làm tham số mặc định) cũng dễ gây bug vì giá trị được giữ lại giữa các lần gọi hàm. Những lỗi này khó phát hiện vì chương trình vẫn chạy bình thường nhưng dữ liệu bị thay đổi ngoài ý muốn.

[EN] Hard-to-detect bugs often happen when mutable objects are shared between multiple variables or functions, especially when using shallow copy with nested data, because a change in one place silently affects another. They also occur with shared state in multi-threading or async code without proper synchronization. In addition, using a mutable default argument in a function (such as a list or dict) can cause bugs because the value persists between function calls. These issues are difficult to detect because the program still runs, but the data changes unexpectedly.

[VN] Decorator hoạt động ra sao?

[EN] How do decorators work?

[VN] Decorator trong Python là một hàm dùng để bọc (wrap) một hàm khác nhằm thêm hành vi trước hoặc sau khi hàm đó chạy mà không sửa code gốc. Khi dùng ký hiệu @decorator, Python sẽ truyền hàm gốc vào decorator và trả về một hàm mới thay thế. Thực chất, decorator hoạt động dựa trên việc hàm là first-class object và closure. Decorator thường được dùng cho logging, authentication, caching hoặc đo thời gian thực thi.

[EN] A decorator in Python is a function that wraps another function to add behavior before or after it runs without modifying the original code. When using the @decorator syntax, Python passes the original function to the decorator and replaces it with a new function returned by the decorator. It works because functions are first-class objects and support closures. Decorators are commonly used for logging, authentication, caching, or measuring execution time.

[VN] Ứng dụng thực tế trong logging, auth, caching?

[EN] Practical applications in logging, authentication, and caching?

[VN] Trong thực tế, decorator thường được dùng cho logging bằng cách ghi lại thông tin trước và sau khi hàm chạy như tham số, thời gian thực thi hoặc lỗi. Với authentication, decorator có thể kiểm tra token hoặc quyền truy cập trước khi cho phép hàm API thực thi. Với caching, decorator có thể lưu kết quả của hàm vào bộ nhớ (ví dụ theo tham số đầu

vào) và trả lại kết quả đã lưu nếu được gọi lại, giúp giảm thời gian xử lý và tải hệ thống. Nhờ đó, decorator giúp tách logic chung ra khỏi business logic và làm code sạch hơn.

[EN] In practice, decorators are often used for logging by recording information before and after a function runs, such as parameters, execution time, or errors. For authentication, a decorator can check a token or user permissions before allowing an API function to execute. For caching, a decorator can store the function's result in memory (based on input arguments) and return the cached result when called again, reducing processing time and system load. This helps separate common logic from business logic and keeps the code cleaner.

[VN] Context manager là gì?

[EN] What is a context manager?

[VN] Context manager trong Python là một cơ chế giúp quản lý tài nguyên tự động, như mở file, kết nối database hoặc lock. Nó đảm bảo tài nguyên được khởi tạo khi bắt đầu và được giải phóng đúng cách khi kết thúc, kể cả khi xảy ra lỗi. Context manager thường được dùng với cú pháp with, và hoạt động dựa trên hai phương thức `__enter__` và `__exit__`. Nhờ đó, code an toàn hơn, tránh quên đóng tài nguyên và giảm rò rỉ bộ nhớ.

[EN] A context manager in Python is a mechanism that automatically manages resources, such as opening files, database connections, or locks. It ensures the resource is properly initialized at the start and correctly released at the end, even if an error occurs. It is commonly used with the with statement and works through the `__enter__` and `__exit__` methods. This makes the code safer, prevents forgetting to release resources, and reduces memory leaks.

[VN] Khi nào nên tự viết context manager?

[EN] When should you write your own context manager?

[VN] Bạn nên tự viết context manager khi cần quản lý tài nguyên hoặc trạng thái mà phải được dọn dẹp chắc chắn sau khi sử dụng, như kết nối database, transaction, lock, file tạm hoặc thay đổi cấu hình tạm thời. Nếu việc giải phóng tài nguyên cần chạy dù có lỗi xảy ra, context manager giúp đảm bảo điều đó một cách an toàn và rõ ràng. Nó cũng hữu ích khi bạn muốn gom logic setup và cleanup vào một chỗ để code dễ đọc và dễ bảo trì hơn.

[EN] You should write a custom context manager when you need to manage a resource or state that must always be cleaned up after use, such as a database connection, transaction, lock, temporary file, or temporary configuration change. If the cleanup must run even when an error occurs, a context manager ensures this in a safe and clear way. It is also

useful to group setup and cleanup logic in one place, making the code more readable and maintainable.

[VN] Metaclass là gì?

[EN] What is a metaclass?

[VN] Metaclass trong Python là “class của class”, nghĩa là nó định nghĩa cách một class được tạo ra. Khi bạn tạo một class, Python thực chất gọi một metaclass (mặc định là type) để xây dựng class đó. Metaclass cho phép bạn can thiệp vào quá trình tạo class, ví dụ như tự động thêm method, kiểm tra cấu trúc class hoặc đăng ký class vào một hệ thống nào đó. Tuy nhiên, metaclass là tính năng nâng cao và chỉ nên dùng khi thật sự cần vì nó làm code phức tạp hơn.

[EN] A metaclass in Python is the “class of a class”, meaning it defines how a class is created. When you define a class, Python actually calls a metaclass (by default type) to build that class. A metaclass allows you to customize the class creation process, for example by automatically adding methods, validating the class structure, or registering the class in a system. However, metaclasses are an advanced feature and should only be used when truly necessary because they make the code more complex.

[VN] Khi nào thực sự cần dùng Metaclass?

[EN] When do you actually need to use a metaclass?

[VN] Bạn thực sự cần dùng metaclass khi muốn kiểm soát hoặc thay đổi cách class được tạo ra trên toàn hệ thống, ví dụ như tự động đăng ký các class con vào một registry, ép buộc class phải có một số method nhất định, hoặc tự động thêm thuộc tính chung cho nhiều class. Metaclass thường được dùng trong framework, ORM hoặc thư viện lớn, nơi cần áp dụng quy tắc chung cho nhiều class. Tuy nhiên, trong hầu hết ứng dụng thông thường, có thể thay thế bằng decorator hoặc class base, nên chỉ dùng metaclass khi các cách đơn giản hơn không đáp ứng được.

[EN] You truly need a metaclass when you want to control or modify how classes are created across the system, such as automatically registering subclasses in a registry, enforcing that a class implements certain methods, or automatically adding shared attributes to many classes. Metaclasses are often used in frameworks, ORMs, or large libraries where common rules must be applied to many classes. However, in most normal applications, they can be replaced by decorators or base classes, so you should use metaclasses only when simpler solutions are not enough.

[VN] Monkey patching là gì?

[EN] What is monkey patching?

[VN] Monkey patching là kỹ thuật thay đổi hoặc ghi đè hành vi của một class hoặc module tại runtime mà không sửa code gốc. Ví dụ, bạn có thể thay thế một method của thư viện bên ngoài bằng một hàm mới để sửa lỗi tạm thời hoặc thay đổi logic. Cách này linh hoạt nhưng có thể gây khó hiểu và khó debug vì hành vi hệ thống bị thay đổi ngầm. Do đó, chỉ nên dùng monkey patching khi thật sự cần và hiểu rõ rủi ro.

[EN] Monkey patching is a technique where you modify or override the behavior of a class or module at runtime without changing the original source code. For example, you can replace a method from an external library with a new function to temporarily fix a bug or change logic. This approach is flexible but can make the system harder to understand and debug because behavior changes silently. Therefore, monkey patching should only be used when truly necessary and with a clear understanding of the risks.

[VN] Khi nào nên và không nên dùng Monkey patching?

[EN] When should and shouldn't monkey patching be used?

[VN] Bạn nên dùng monkey patching khi cần sửa lỗi khẩn cấp trong thư viện bên thứ ba mà chưa thể cập nhật phiên bản mới, hoặc khi cần thay đổi hành vi tạm thời trong môi trường test hoặc prototype. Tuy nhiên, bạn không nên dùng monkey patching trong production lâu dài vì nó làm code khó hiểu, khó bảo trì và dễ gây lỗi khó phát hiện do thay đổi hành vi ngầm. Nếu có thể, nên ưu tiên cách rõ ràng hơn như kế thừa (inheritance), wrapper hoặc cấu hình chính thức từ thư viện.

[EN] You should use monkey patching when you need to quickly fix a bug in a third-party library and cannot upgrade it yet, or when you need to temporarily change behavior in testing or prototyping. However, you should not use monkey patching as a long-term solution in production because it makes the code harder to understand, maintain, and debug due to hidden behavior changes. If possible, prefer clearer solutions such as inheritance, wrappers, or official configuration provided by the library.

[VN] Python import system hoạt động thế nào?

[EN] How does the Python import system work?

[VN] Python import system hoạt động bằng cách tìm module theo thứ tự trong danh sách sys.path, bao gồm thư mục hiện tại, thư mục cài đặt chuẩn và các thư mục được cấu hình thêm. Khi bạn import một module, Python sẽ kiểm tra xem module đó đã được load trong sys.modules chưa; nếu rồi thì dùng lại, nếu chưa thì tìm file tương ứng (.py, package, hoặc compiled file), thực thi code trong module một lần và lưu vào sys.modules để tái sử dụng. Vì vậy, mỗi module chỉ được thực thi một lần trong một process, giúp tăng hiệu năng và tránh load lại nhiều lần.

[EN] The Python import system works by searching for a module in the order listed in `sys.path`, which includes the current directory, standard library directories, and any additional configured paths. When you import a module, Python first checks if it is already loaded in `sys.modules`; if yes, it reuses it, if not, it finds the corresponding file (`.py`, package, or compiled file), executes the module code once, and stores it in `sys.modules` for future use. This means each module is executed only once per process, improving performance and avoiding repeated loading.

[VN] Circular import xử lý ra sao?

[EN] How are circular imports handled?

[VN] Circular import xảy ra khi hai hoặc nhiều module import lẫn nhau, làm cho một module chưa được khởi tạo xong đã bị module khác sử dụng, gây lỗi hoặc thiếu thuộc tính. Cách xử lý phổ biến là di chuyển câu lệnh import vào bên trong hàm hoặc method để trì hoãn việc import cho đến khi thực sự cần, tách phần code chung sang một module riêng để cả hai cùng import, hoặc thiết kế lại cấu trúc project để giảm phụ thuộc vòng. Mục tiêu là phá vỡ vòng phụ thuộc và làm cho thứ tự import rõ ràng hơn.

[EN] Circular import happens when two or more modules import each other, causing one module to be used before it is fully initialized, which can lead to errors or missing attributes. A common solution is to move the import statement inside a function or method to delay the import until it is actually needed, extract shared code into a separate module that both can import, or redesign the project structure to reduce circular dependencies. The goal is to break the dependency cycle and make the import order clearer.

[VN] Typing (type hints) giúp gì trong dự án lớn?

[EN] How do typing (type hints) help in large projects?

[VN] Typing (type hints) trong Python giúp mô tả rõ kiểu dữ liệu của biến, tham số và giá trị trả về, từ đó làm code dễ đọc và dễ hiểu hơn trong dự án lớn. Nó cho phép dùng công cụ như `mypy` hoặc IDE để kiểm tra lỗi kiểu dữ liệu trước khi chạy, giảm bug do truyền sai kiểu. Type hints cũng hỗ trợ refactor an toàn hơn và giúp các thành viên trong team hiểu API và cấu trúc dữ liệu nhanh hơn. Dù không bắt buộc khi chạy chương trình, type hints rất hữu ích để tăng chất lượng và độ tin cậy của code.

[EN] Typing (type hints) in Python helps describe the data types of variables, parameters, and return values, making the code more readable and understandable in large projects. It allows tools like `mypy` or IDEs to detect type errors before runtime, reducing bugs caused by wrong data types. Type hints also make refactoring safer and help team members understand APIs and data structures more quickly. Although not required for program execution, type hints are very useful for improving code quality and reliability.

[VN] Khi nào nên dùng mypy?

[EN] When should you use mypy?

[VN] Bạn nên dùng mypy khi dự án lớn, có nhiều module và nhiều người cùng phát triển, để kiểm tra lỗi kiểu dữ liệu trước khi chạy chương trình. Mypy đặc biệt hữu ích khi code có nhiều logic phức tạp, nhiều tham số và kiểu dữ liệu lồng nhau, vì nó giúp phát hiện lỗi sớm và hỗ trợ refactor an toàn hơn. Nó cũng nên dùng khi bạn đã áp dụng type hints một cách nghiêm túc và muốn đảm bảo tính nhất quán trong toàn bộ codebase. Với dự án nhỏ hoặc script đơn giản, mypy có thể không cần thiết.

[EN] You should use mypy in large projects with many modules and multiple developers, to catch type errors before running the program. It is especially useful when the code has complex logic, many parameters, and nested data types, because it helps detect issues early and makes refactoring safer. It is also recommended when you seriously adopt type hints and want to ensure consistency across the entire codebase. For small projects or simple scripts, mypy may not be necessary.

[VN] Dataclass vs Pydantic vs NamedTuple khác nhau thế nào?

[EN] How do Dataclass, Pydantic, and NamedTuple differ?

[VN] Dataclass là cách đơn giản để tạo class chứa dữ liệu với ít code, tự động sinh ra `__init__`, `__repr__`, và có thể mutable hoặc immutable; nó phù hợp cho model nội bộ và logic thuần Python. Pydantic là thư viện dùng để định nghĩa model có validation và parsing dữ liệu mạnh mẽ, tự động kiểm tra và chuyển đổi kiểu, rất phù hợp cho API hoặc xử lý dữ liệu đầu vào. NamedTuple là cấu trúc dữ liệu nhẹ, immutable, truy cập bằng tên thuộc tính nhưng hoạt động gần giống tuple, phù hợp khi cần cấu trúc đơn giản, không cần validation và muốn tối ưu bộ nhớ.

[EN] Dataclass is a simple way to create data classes with less boilerplate, automatically generating methods like `__init__` and `__repr__`, and it can be mutable or immutable; it is suitable for internal models and pure Python logic. Pydantic is a library used to define models with strong data validation and parsing, automatically checking and converting types, which is very suitable for APIs or input data handling. NamedTuple is a lightweight, immutable data structure that allows attribute access by name but behaves like a tuple, suitable when you need a simple structure without validation and want better memory efficiency.

[VN] Concurrency trong Python gồm những mô hình nào?

[EN] What concurrency models exist in Python?

[VN] Concurrency trong Python gồm nhiều mô hình chính như threading, multiprocessing và asynchronous (async/await). Threading cho phép nhiều luồng chạy song song trong cùng một process nhưng bị giới hạn bởi GIL nên phù hợp với I/O-bound task. Multiprocessing tạo nhiều process riêng biệt, mỗi process có bộ nhớ riêng và không bị GIL giới hạn, phù hợp với CPU-bound task. Asynchronous dùng event loop và async/await để xử lý nhiều tác vụ I/O không đồng bộ trong một thread, giúp tối ưu hiệu năng cho hệ thống web hoặc API có nhiều request đồng thời.

[EN] Concurrency in Python includes several main models such as threading, multiprocessing, and asynchronous programming (async/await). Threading allows multiple threads to run in the same process but is limited by the GIL, so it is suitable for I/O-bound tasks. Multiprocessing creates separate processes with their own memory and is not limited by the GIL, making it good for CPU-bound tasks. Asynchronous programming uses an event loop and async/await to handle many non-blocking I/O tasks in a single thread, improving performance for web systems or APIs with many concurrent requests.

[VN] Async IO vs threading vs multiprocessing khác nhau ra sao trong thực tế?

[EN] How do Async IO, threading, and multiprocessing differ in practice?

[VN] Trong thực tế, Async IO phù hợp nhất cho I/O-bound task như gọi API, đọc file hoặc truy vấn database vì nó dùng event loop và chạy trong một thread, tiết kiệm tài nguyên và xử lý nhiều request cùng lúc, thường dùng trong web server như FastAPI. Threading cũng phù hợp cho I/O-bound task nhưng mỗi thread vẫn tốn tài nguyên hệ thống và bị giới hạn bởi GIL nên không tăng tốc tốt cho CPU-bound task. Multiprocessing tạo nhiều process riêng biệt, không bị GIL giới hạn nên phù hợp cho CPU-bound task như xử lý dữ liệu lớn hoặc tính toán nặng, nhưng tốn nhiều bộ nhớ và chi phí tạo process cao hơn. Vì vậy, chọn mô hình phụ thuộc vào việc bạn xử lý I/O hay CPU và yêu cầu tài nguyên hệ thống.

[EN] In practice, Async IO is best for I/O-bound tasks such as calling APIs, reading files, or querying databases because it uses an event loop in a single thread, saves resources, and handles many requests at the same time, commonly used in web servers like FastAPI. Threading is also suitable for I/O-bound tasks, but each thread consumes system resources and is limited by the GIL, so it does not improve CPU-bound performance. Multiprocessing creates separate processes that are not limited by the GIL, making it suitable for CPU-bound tasks like heavy computation or large data processing, but it uses more memory and has higher process creation cost. Therefore, the choice depends on whether the task is I/O-bound or CPU-bound and on system resource requirements.

[VN] Event loop hoạt động như thế nào?

[EN] How does the event loop work?

[VN] Event loop là cơ chế trung tâm trong Async IO, hoạt động bằng cách chạy liên tục trong một vòng lặp để theo dõi và xử lý các tác vụ bất đồng bộ. Khi một coroutine gặp thao tác I/O như gọi API hoặc đọc file, nó sẽ “nhường quyền” và đăng ký callback với event loop, thay vì chờ đợi và chặn chương trình. Event loop sẽ chuyển sang chạy tác vụ khác, và khi I/O hoàn tất, nó đưa coroutine cũ trở lại hàng đợi để tiếp tục thực thi. Nhờ đó, một thread có thể xử lý nhiều tác vụ I/O cùng lúc một cách hiệu quả.

[EN] The event loop is the core mechanism in Async IO, running continuously in a loop to monitor and execute asynchronous tasks. When a coroutine reaches an I/O operation such as an API call or file read, it “yields control” and registers a callback with the event loop instead of blocking the program. The event loop then switches to other tasks, and when the I/O is complete, it puts the original coroutine back in the queue to continue execution. This allows a single thread to efficiently handle many I/O tasks at the same time.

[VN] Blocking call ảnh hưởng gì đến async app?

[EN] How do blocking calls affect an async app?

[VN] Trong một async app, blocking call sẽ chặn event loop và làm toàn bộ ứng dụng bị đứng tạm thời cho đến khi thao tác đó hoàn thành. Vì event loop chỉ chạy trên một thread, nếu có một hàm chạy lâu và không “nhường quyền”, các request khác sẽ không được xử lý, làm giảm performance và khả năng xử lý đồng thời. Do đó, trong async app nên tránh dùng hàm blocking (như sleep thường, I/O đồng bộ hoặc tính toán nặng) hoặc phải chuyển chúng sang thread pool hay process pool.

[EN] In an async app, a blocking call will block the event loop and pause the whole application until that operation finishes. Since the event loop runs in a single thread, if one function runs for a long time without yielding control, other requests cannot be processed, reducing performance and concurrency. Therefore, in async applications you should avoid blocking functions (such as normal sleep, synchronous I/O, or heavy computation) or move them to a thread pool or process pool.

[VN] Làm sao detect memory leak trong Python?

[EN] How to detect memory leaks in Python?

[VN] Để detect memory leak trong Python, bạn có thể theo dõi mức sử dụng bộ nhớ theo thời gian bằng các công cụ như tracemalloc, gc hoặc profiler như memory_profiler. Nếu bộ nhớ tăng liên tục mà không giảm sau khi xử lý xong task, đó có thể là dấu hiệu leak. Bạn cũng có thể kiểm tra object nào chưa được giải phóng bằng cách dùng gc.get_objects() hoặc xem reference count để phát hiện vòng tham chiếu. Ngoài ra, monitor bằng công cụ hệ thống như top hoặc container metrics trong production cũng giúp phát hiện bất thường sớm.

[EN] To detect memory leaks in Python, you can monitor memory usage over time using tools like tracemalloc, gc, or profilers such as memory_profiler. If memory keeps increasing and does not drop after tasks finish, it may indicate a leak. You can also inspect which objects are not released by using gc.get_objects() or checking reference counts to find circular references. In production, monitoring system tools like top or container metrics also helps detect abnormal memory growth early.

[VN] Profiling Python code bằng cProfile hoặc line_profiler như thế nào?

[EN] How to profile Python code using cProfile or line_profiler?

[VN] Để profiling Python bằng cProfile, bạn có thể chạy chương trình với lệnh `python -m cProfile script.py` hoặc dùng `cProfile.run()` trong code để xem hàm nào tốn nhiều thời gian nhất; kết quả cho biết số lần gọi và tổng thời gian thực thi của từng hàm. Với line_profiler, bạn cài thư viện, thêm decorator `@profile` vào hàm cần phân tích, rồi chạy bằng lệnh `kernprof -l script.py` để xem thời gian thực thi chi tiết theo từng dòng code. cProfile phù hợp để xem tổng quan hiệu năng toàn chương trình, còn line_profiler giúp phân tích chi tiết từng dòng khi cần tối ưu sâu.

[EN] To profile Python code with cProfile, you can run the program using `python -m cProfile script.py` or call `cProfile.run()` inside the code to see which functions consume the most time; the result shows the number of calls and total execution time per function. With line_profiler, you install the library, add the `@profile` decorator to the target function, and run `kernprof -l script.py` to get detailed timing for each line of code. cProfile is useful for an overall performance view, while line_profiler helps analyze specific lines when deep optimization is needed.

[VN] Time complexity ảnh hưởng thế nào đến API performance?

[EN] How does time complexity affect API performance?

[VN] Time complexity ảnh hưởng trực tiếp đến API performance vì nó quyết định thời gian xử lý tăng như thế nào khi dữ liệu lớn hơn. Nếu thuật toán có độ phức tạp $O(n)$ hoặc $O(n \log n)$, thời gian tăng tuyến tính hoặc gần tuyến tính và thường vẫn chấp nhận được; nhưng nếu là $O(n^2)$ hoặc cao hơn, thời gian xử lý sẽ tăng rất nhanh khi số lượng dữ liệu hoặc request tăng, làm API chậm và dễ quá tải. Trong hệ thống thực tế, time complexity kém có thể gây tăng CPU, tăng latency và giảm throughput, đặc biệt khi traffic cao hoặc dữ liệu lớn.

[EN] Time complexity directly affects API performance because it determines how processing time grows as data size increases. If an algorithm has $O(n)$ or $O(n \log n)$ complexity, the time grows linearly or near linearly and is usually acceptable; but with

$O(n^2)$ or higher, execution time increases very quickly as data or request volume grows, making the API slow and overloaded. In real systems, poor time complexity can lead to high CPU usage, increased latency, and reduced throughput, especially under high traffic or large datasets.

[VN] Thiết kế project structure cho ứng dụng lớn như thế nào?

[EN] How to design project structure for a large application?

[VN] Khi thiết kế project structure cho ứng dụng lớn, bạn nên chia theo domain hoặc feature thay vì chỉ chia theo loại file, ví dụ mỗi module có controller, service, repository và model riêng để dễ mở rộng và bảo trì. Cần tách rõ layer như presentation, business logic và data access để giảm coupling và tăng testability. Ngoài ra, nên có thư mục riêng cho config, common utilities, tests và scripts, đồng thời đặt tên rõ ràng, tránh circular import và giữ dependency một chiều. Mục tiêu là cấu trúc dễ hiểu, dễ scale và dễ cho nhiều người cùng làm việc.

[EN] When designing a project structure for a large application, you should organize it by domain or feature instead of only by file type, for example each module contains its own controller, service, repository, and model to make scaling and maintenance easier. You should clearly separate layers such as presentation, business logic, and data access to reduce coupling and improve testability. It is also important to have separate folders for config, common utilities, tests, and scripts, use clear naming, avoid circular imports, and keep one-way dependencies. The goal is to create a structure that is easy to understand, scalable, and suitable for team collaboration.

[VN] Làm sao tách business logic khỏi framework (clean architecture)?

[EN] How to separate business logic from the framework (clean architecture)?

[VN] Để tách business logic khỏi framework theo clean architecture, bạn cần đặt toàn bộ logic nghiệp vụ vào các layer độc lập như domain hoặc use case, không phụ thuộc trực tiếp vào framework, database hay thư viện bên ngoài. Framework chỉ nên nằm ở outer layer và đóng vai trò adapter để nhận request, gọi vào use case và trả response. Bạn nên dùng interface hoặc abstraction để business layer không biết chi tiết implement của database hay API bên ngoài. Nhờ vậy, core logic có thể test độc lập, dễ thay framework và dễ bảo trì lâu dài.

[EN] To separate business logic from the framework using clean architecture, you should place all business rules inside independent layers such as domain or use case, without directly depending on the framework, database, or external libraries. The framework should stay in the outer layer and act as an adapter to receive requests, call the use case, and return responses. You should use interfaces or abstractions so the business layer does

not know the implementation details of the database or external APIs. This allows the core logic to be tested independently, makes it easier to replace frameworks, and improves long-term maintainability.

[VN] Dependency injection trong Python thuần (không dùng FastAPI) làm thế nào?

[EN] How to implement dependency injection in pure Python (without FastAPI)?

[VN] Dependency Injection trong Python thuần có thể thực hiện bằng cách truyền dependency qua constructor hoặc tham số hàm thay vì tạo trực tiếp bên trong class. Ví dụ, một service không tự khởi tạo database mà nhận database object từ bên ngoài, giúp giảm coupling và dễ test bằng cách truyền mock object. Bạn cũng có thể dùng factory function hoặc một container đơn giản để quản lý việc tạo và cung cấp dependency. Cách này giúp code linh hoạt hơn, dễ thay đổi implementation và dễ kiểm thử.

[EN] Dependency Injection in pure Python can be done by passing dependencies through the constructor or function parameters instead of creating them directly inside the class. For example, a service should receive a database object from outside instead of instantiating it internally, which reduces coupling and makes testing easier by injecting mock objects. You can also use a factory function or a simple container to manage object creation and dependency wiring. This approach makes the code more flexible, easier to replace implementations, and easier to test.

[VN] Khi nào nên dùng class, khi nào nên dùng function thuần?

[EN] When should you use a class, and when should you use a plain function?

[VN] Bạn nên dùng class khi cần quản lý state (trạng thái) hoặc dữ liệu liên quan và nhiều hành vi hoạt động trên cùng dữ liệu đó, ví dụ như model, service hoặc đối tượng có vòng đời rõ ràng. Class cũng phù hợp khi cần kế thừa, đóng gói và mở rộng hành vi. Ngược lại, nên dùng function thuần khi logic đơn giản, không cần lưu trạng thái giữa các lần gọi và chỉ xử lý input để trả về output; cách này giúp code dễ đọc, dễ test và ít phức tạp hơn. Tóm lại, nếu cần state và cấu trúc rõ ràng thì dùng class, nếu chỉ cần xử lý logic độc lập thì dùng function.

[EN] You should use a class when you need to manage state or related data with multiple behaviors operating on that data, such as models, services, or objects with a clear lifecycle. Classes are also useful when you need inheritance, encapsulation, and extensibility. On the other hand, you should use pure functions when the logic is simple, does not require shared state between calls, and only transforms input into output; this makes the code easier to read, test, and maintain. In short, use a class when you need state and structure, and use functions for independent, stateless logic.

[VN] Làm sao viết code Python thread-safe?

[EN] How to write thread-safe Python code?

[VN] Để viết code Python thread-safe, bạn cần tránh chia sẻ mutable state giữa nhiều thread nếu không cần thiết, hoặc bảo vệ vùng dữ liệu chung bằng cơ chế đồng bộ như Lock, RLock hoặc Semaphore trong module threading. Bạn nên giữ critical section nhỏ nhất có thể để giảm tranh chấp tài nguyên và tránh deadlock. Ngoài ra, có thể dùng queue để giao tiếp giữa các thread thay vì truy cập trực tiếp dữ liệu chung, hoặc chuyển sang multiprocessing nếu cần xử lý CPU-bound. Thiết kế immutable object và hạn chế global variable cũng giúp giảm lỗi race condition.

[EN] To write thread-safe Python code, you should avoid sharing mutable state between threads when possible, or protect shared data using synchronization mechanisms such as Lock, RLock, or Semaphore from the threading module. Keep the critical section as small as possible to reduce resource contention and avoid deadlocks. You can also use a queue for communication between threads instead of directly accessing shared data, or switch to multiprocessing for CPU-bound tasks. Designing immutable objects and minimizing global variables also helps reduce race condition issues.

[VN] Gunicorn vs Uvicorn khác nhau thế nào?

[EN] What is the difference between Gunicorn and Uvicorn?

[VN] Gunicorn là một WSGI server truyền thống, thường dùng cho ứng dụng synchronous như Flask hoặc Django (không async). Nó quản lý nhiều worker process để xử lý request song song nhưng không hỗ trợ async native. Uvicorn là một ASGI server, được thiết kế cho ứng dụng async như FastAPI hoặc Django ASGI, hỗ trợ event loop và xử lý I/O bất đồng bộ hiệu quả hơn. Trong thực tế, Gunicorn phù hợp cho app sync ổn định, còn Uvicorn phù hợp cho app async; đôi khi người ta chạy Uvicorn worker bên trong Gunicorn để kết hợp quản lý process và khả năng async.

[EN] Gunicorn is a traditional WSGI server, commonly used for synchronous applications like Flask or Django (without async). It manages multiple worker processes to handle requests in parallel but does not support native async. Uvicorn is an ASGI server designed for async applications such as FastAPI or Django ASGI, supporting the event loop and handling asynchronous I/O more efficiently. In practice, Gunicorn is suitable for stable sync apps, while Uvicorn is better for async apps; sometimes Uvicorn workers are run inside Gunicorn to combine process management with async capability.

[VN] Worker type ảnh hưởng ra sao?

[EN] How does worker type affect performance?

[VN] Worker type ảnh hưởng trực tiếp đến cách ứng dụng xử lý request, mức độ sử dụng CPU và khả năng chịu tải. Ví dụ, sync worker xử lý từng request theo kiểu blocking nên phù hợp với ứng dụng I/O đơn giản nhưng có thể bị chậm nếu gặp request lâu; async worker có thể xử lý nhiều request cùng lúc khi chờ I/O, phù hợp với API gọi nhiều service bên ngoài; còn worker dạng process giúp tận dụng nhiều CPU core cho tác vụ nặng. Chọn sai worker type có thể làm tăng latency, giảm throughput hoặc lãng phí tài nguyên hệ thống.

[EN] Worker type directly affects how the application handles requests, CPU usage, and scalability. For example, a sync worker processes requests in a blocking way and is suitable for simple I/O apps but can become slow when handling long requests; an async worker can handle many requests concurrently while waiting for I/O, which is good for APIs calling external services; a process-based worker helps use multiple CPU cores for heavy tasks. Choosing the wrong worker type can increase latency, reduce throughput, or waste system resources.

[VN] WSGI vs ASGI khác nhau thế nào?

[EN] What is the difference between WSGI and ASGI?

[VN] WSGI là chuẩn giao tiếp giữa web server và ứng dụng Python theo mô hình synchronous, mỗi request được xử lý theo kiểu blocking và phù hợp với ứng dụng truyền thống như Flask hoặc Django (phiên bản sync). ASGI là chuẩn mới hỗ trợ asynchronous, cho phép xử lý nhiều request cùng lúc, hỗ trợ WebSocket và background task tốt hơn. Vì vậy, WSGI phù hợp cho app đơn giản, ít I/O phức tạp, còn ASGI phù hợp cho ứng dụng hiện đại cần async, realtime hoặc hiệu năng cao khi xử lý nhiều kết nối đồng thời.

[EN] WSGI is a standard interface between a web server and a Python application using a synchronous model, where each request is handled in a blocking way and is suitable for traditional apps like Flask or synchronous Django. ASGI is a newer standard that supports asynchronous programming, allowing multiple requests to be handled concurrently and better support for WebSocket and background tasks. Therefore, WSGI is suitable for simple apps with less complex I/O, while ASGI is better for modern applications that require async, real-time features, or high performance with many concurrent connections.

[VN] Làm sao scale Python app khi GIL tồn tại?

[EN] How to scale a Python app when the GIL exists?

[VN] Khi GIL tồn tại, một process Python chỉ chạy được một thread thực thi bytecode tại một thời điểm, nên với tác vụ CPU-bound bạn nên scale bằng cách dùng multiprocessing hoặc chạy nhiều process (ví dụ qua Gunicorn) để tận dụng nhiều CPU core. Với tác vụ I/O-bound, có thể dùng async hoặc threading vì khi chờ I/O thì GIL được nhả ra. Ngoài ra,

có thể scale ngang bằng cách chạy nhiều instance ứng dụng sau load balancer, hoặc dùng service bên ngoài cho tác vụ nặng như queue và worker riêng. Cách chọn phụ thuộc vào loại workload là CPU-bound hay I/O-bound.

[EN] Because of the GIL, one Python process can execute only one thread of bytecode at a time, so for CPU-bound tasks you should scale using multiprocessing or multiple processes (for example with Gunicorn) to use multiple CPU cores. For I/O-bound tasks, async or threading works well because the GIL is released while waiting for I/O. You can also scale horizontally by running multiple app instances behind a load balancer, or move heavy tasks to external services like queues with separate workers. The right approach depends on whether the workload is CPU-bound or I/O-bound.

[VN] Khi nào cần dùng C extension hoặc Cython?

[EN] When should you use C extensions or Cython?

[VN] Bạn cần dùng C extension hoặc Cython khi ứng dụng có đoạn code CPU-bound chạy chậm và profiling cho thấy đó là bottleneck quan trọng mà tối ưu bằng Python thuần không đủ. Ví dụ như xử lý số học lớn, vòng lặp tính toán nhiều lần, xử lý dữ liệu lớn hoặc thuật toán phức tạp cần hiệu năng cao. Viết phần nặng bằng C hoặc Cython giúp giảm overhead của Python và tận dụng tốc độ của C. Tuy nhiên, chỉ nên dùng khi thật sự cần vì làm tăng độ phức tạp build, debug và bảo trì.

[EN] You should use a C extension or Cython when your application has CPU-bound code that runs slowly and profiling shows it is a major bottleneck that cannot be optimized enough with pure Python. Examples include heavy numerical computation, tight loops, large data processing, or complex algorithms that require high performance. Writing the heavy part in C or Cython reduces Python overhead and takes advantage of C speed. However, you should only use this approach when necessary because it increases build complexity, debugging difficulty, and maintenance cost.

[VN] Mock vs patch khác nhau thế nào?

[EN] What is the difference between mock and patch?

[VN] Mock là một object giả được tạo ra để mô phỏng hành vi của dependency trong quá trình test, giúp bạn kiểm soát giá trị trả về và kiểm tra cách nó được gọi. Patch là kỹ thuật thay thế tạm thời một object thật (ví dụ function, class, module) bằng mock trong phạm vi test, thường dùng trong unittest.mock để “đổi” dependency tại đúng nơi nó được import. Nói đơn giản, mock là object giả, còn patch là cách bạn chèn object giả đó vào vị trí cần test.

[EN] A mock is a fake object created to simulate the behavior of a dependency during testing, allowing you to control return values and verify how it was called. Patch is a technique that temporarily replaces a real object (such as a function, class, or module) with a mock within the scope of a test, commonly using `unittest.mock` to override the dependency where it is imported. In simple terms, a mock is the fake object, while patch is the way you insert that fake object into the place you want to test.

[VN] Test async function như thế nào?

[EN] How to test an async function?

[VN] Để test async function trong Python, bạn cần dùng test framework hỗ trợ async như `pytest` với `pytest-asyncio` hoặc `unittest` với `IsolatedAsyncioTestCase`. Thay vì gọi trực tiếp function, bạn phải dùng `await` bên trong một async test function. Nếu function có gọi dependency async khác, bạn có thể dùng `AsyncMock` để mock chúng. Quan trọng là đảm bảo event loop được quản lý đúng cách và test không bị blocking bởi code đồng bộ.

[EN] To test an async function in Python, you should use a testing framework that supports async, such as `pytest` with `pytest-asyncio` or `unittest` with `IsolatedAsyncioTestCase`. Instead of calling the function directly, you need to use `await` inside an async test function. If the function calls other async dependencies, you can mock them using `AsyncMock`. It is important to ensure the event loop is properly managed and that the test is not blocked by synchronous code.

[VN] Property-based testing là gì?

[EN] What is property-based testing?

[VN] Property-based testing là cách kiểm thử trong đó bạn không chỉ viết test cho một vài input cụ thể, mà định nghĩa các “tính chất” (property) mà hàm phải luôn đúng, rồi framework sẽ tự sinh nhiều dữ liệu ngẫu nhiên để kiểm tra. Ví dụ, nếu bạn viết hàm `sort`, property có thể là “kết quả luôn có cùng số phần tử và được sắp xếp tăng dần”. Cách này giúp phát hiện edge case mà bạn có thể không nghĩ tới khi viết test thủ công.

[EN] Property-based testing is a testing approach where you do not only write tests for a few specific inputs, but instead define general “properties” that the function must always satisfy, and the framework automatically generates many random inputs to verify them. For example, if you write a `sort` function, a property could be “the result always has the same number of elements and is sorted in ascending order”. This approach helps find edge cases that you might not think of when writing manual tests.

[VN] Coverage cao có đảm bảo code tốt không?

[EN] Does high coverage guarantee good code?

[VN] Coverage cao không đảm bảo code tốt. Coverage chỉ cho biết bao nhiêu phần trăm dòng code đã được chạy trong quá trình test, nhưng không đảm bảo rằng các test đó kiểm tra đúng logic, đủ edge case hay phát hiện bug. Bạn có thể đạt 100% coverage nhưng test chỉ gọi hàm mà không assert đúng giá trị quan trọng. Code tốt cần test có chất lượng, kiểm tra hành vi quan trọng, xử lý lỗi và các trường hợp đặc biệt, chứ không chỉ đạt con số coverage cao.

[EN] High coverage does not guarantee good code. Coverage only shows what percentage of lines were executed during testing, but it does not ensure that the tests check the correct logic, cover important edge cases, or detect bugs. You can reach 100% coverage but still have weak tests that only call functions without asserting meaningful results. Good code requires high-quality tests that verify important behavior, error handling, and edge cases, not just a high coverage number.

[VN] Closure là gì và được dùng để làm gì?

[EN] What is a closure and what is it used for?

[VN] Closure trong Python là một function được tạo bên trong một function khác và nhớ được các biến từ scope bên ngoài, ngay cả khi function bên ngoài đã kết thúc. Điều này xảy ra khi function bên trong sử dụng biến của function bên ngoài. Closure thường được dùng để giữ state, tạo function tùy chỉnh (function factory), hoặc implement decorator mà không cần dùng class. Nó giúp code gọn hơn và cho phép đóng gói dữ liệu cùng với logic xử lý.

[EN] A closure in Python is a function defined inside another function that remembers variables from the outer scope, even after the outer function has finished executing. This happens when the inner function uses variables from the outer function. Closures are often used to keep state, create custom functions (function factories), or implement decorators without using classes. It helps keep the code simple and allows data and behavior to be bundled together.

[VN] NumPy là gì? Tại sao nên dùng NumPy thay vì Python list?

[EN] What is NumPy? Why should we use NumPy instead of Python lists?

[VN] NumPy là một thư viện phổ biến trong Python dùng để xử lý tính toán số học và làm việc với dữ liệu dạng mảng nhiều chiều gọi là ndarray. NumPy được tối ưu bằng C nên các phép toán trên mảng chạy rất nhanh. So với Python list, NumPy hiệu quả hơn vì nó lưu dữ liệu liên tục trong bộ nhớ, chỉ dùng một kiểu dữ liệu cho toàn bộ mảng và hỗ trợ các phép toán vectorized (thực hiện phép toán trên toàn bộ mảng mà không cần vòng lặp). Vì vậy NumPy thường được dùng trong data processing, scientific computing và machine learning khi cần xử lý dữ liệu lớn và tính toán nhanh.

[EN] NumPy is a popular Python library used for numerical computing and working with multi-dimensional arrays called ndarray. NumPy is implemented in C, so operations on arrays run very fast. Compared to Python lists, NumPy is more efficient because data is stored in contiguous memory, all elements have the same data type, and it supports vectorized operations (performing operations on the whole array without loops). For this reason, NumPy is commonly used in data processing, scientific computing, and machine learning where large datasets and fast calculations are required.

[VN] ndarray trong NumPy là gì? Nó khác gì với list trong Python?

[EN] What is an ndarray in NumPy? How is it different from a Python list?

[VN] Trong NumPy, ndarray (N-dimensional array) là cấu trúc dữ liệu chính dùng để lưu trữ và xử lý dữ liệu dạng mảng nhiều chiều như vector, matrix hoặc tensor. ndarray khác với Python list ở chỗ tất cả phần tử trong ndarray phải có cùng kiểu dữ liệu (homogeneous), dữ liệu được lưu liên tục trong bộ nhớ nên truy cập và tính toán nhanh hơn, và nó hỗ trợ các phép toán vectorized trên toàn bộ mảng mà không cần vòng lặp. Trong khi đó, Python list có thể chứa nhiều kiểu dữ liệu khác nhau, dữ liệu không được lưu liên tục trong bộ nhớ và thường cần dùng loop để thực hiện phép toán nên chậm hơn khi xử lý dữ liệu lớn.

[EN] In NumPy, ndarray (N-dimensional array) is the main data structure used to store and process multi-dimensional data such as vectors, matrices, or tensors. ndarray is different from a Python list because all elements in an ndarray must have the same data type (homogeneous), the data is stored in contiguous memory which makes access and computation faster, and it supports vectorized operations on the whole array without using loops. In contrast, a Python list can contain different data types, its data is not stored contiguously in memory, and it usually requires loops for calculations, which makes it slower for large data processing.

[VN] Vectorization trong NumPy là gì? Vì sao nó nhanh hơn loop Python?

[EN] What is vectorization in NumPy? Why is it faster than Python loops?

[VN] Trong NumPy, vectorization là kỹ thuật thực hiện phép toán trên toàn bộ mảng (ndarray) cùng lúc thay vì lặp qua từng phần tử bằng vòng lặp Python. Ví dụ, ta có thể cộng hai mảng trực tiếp thay vì dùng for. Vectorization nhanh hơn vì các phép toán được thực thi bằng code tối ưu viết bằng C bên trong NumPy, dữ liệu được lưu liên tục trong bộ nhớ và tránh được overhead của vòng lặp Python. Nhờ đó việc xử lý dữ liệu lớn và tính toán số học sẽ nhanh và hiệu quả hơn.

[EN] In NumPy, vectorization is the technique of performing operations on an entire array (ndarray) at once instead of iterating through each element using Python loops. For example, we can add two arrays directly without using a for loop. Vectorization is faster because the operations are executed by highly optimized C code inside NumPy, the data is stored in contiguous memory, and it avoids the overhead of Python loops. This makes numerical computation and large data processing much more efficient.

[VN] Broadcasting trong NumPy là gì?

[EN] What is broadcasting in NumPy?

[VN] Trong NumPy, broadcasting là cơ chế cho phép thực hiện phép toán giữa các mảng có kích thước (shape) khác nhau mà không cần phải sao chép dữ liệu thủ công. NumPy sẽ tự động “mở rộng” mảng nhỏ hơn để khớp với kích thước của mảng lớn hơn theo một số quy tắc về shape, sau đó thực hiện phép toán như cộng, trừ, nhân hoặc chia. Nhờ broadcasting, ta có thể viết code ngắn gọn và thực hiện các phép tính trên dữ liệu lớn một cách hiệu quả mà không cần dùng vòng lặp.

[EN] In NumPy, broadcasting is a mechanism that allows operations between arrays with different shapes without manually copying data. NumPy automatically “expands” the smaller array to match the shape of the larger array based on specific broadcasting rules, and then performs operations such as addition, subtraction, multiplication, or division. This feature helps write shorter code and perform efficient computations on large datasets without using loops.

[VN] Shape và dtype của một array là gì?

[EN] What are the shape and dtype of an array?

[VN] Trong NumPy, shape và dtype là hai thuộc tính quan trọng của một ndarray. Shape cho biết kích thước và cấu trúc của mảng, tức là số chiều và số phần tử ở mỗi chiều (ví dụ (3, 4) nghĩa là 3 hàng và 4 cột). dtype (data type) cho biết kiểu dữ liệu của các phần tử trong mảng, ví dụ như int, float, hoặc bool. Vì tất cả phần tử trong ndarray phải có cùng kiểu dữ liệu, dtype giúp NumPy quản lý bộ nhớ hiệu quả và thực hiện các phép tính nhanh hơn.

[EN] In NumPy, shape and dtype are two important attributes of an ndarray. Shape describes the size and structure of the array, meaning the number of dimensions and the number of elements in each dimension (for example (3, 4) means 3 rows and 4 columns). dtype (data type) defines the type of data stored in the array elements, such as int, float, or bool. Since all elements in an ndarray must have the same data type, dtype helps NumPy manage memory efficiently and perform computations faster.

[VN] Sự khác nhau giữa reshape() và resize() trong NumPy?

[EN] What is the difference between reshape() and resize() in NumPy?

[VN] Trong NumPy, reshape() và resize() đều dùng để thay đổi kích thước (shape) của một ndarray, nhưng cách hoạt động khác nhau. reshape() tạo ra một mảng mới với shape mới mà không thay đổi dữ liệu gốc và thường chỉ tạo một view nếu có thể, vì vậy array ban đầu vẫn giữ nguyên. Ngược lại, resize() thay đổi trực tiếp array gốc và có thể thêm hoặc cắt bớt phần tử nếu kích thước mới khác với kích thước ban đầu. Vì vậy reshape() thường an toàn hơn khi chỉ muốn thay đổi cách nhìn dữ liệu, còn resize() dùng khi thật sự muốn thay đổi kích thước của mảng.

[EN] In NumPy, both reshape() and resize() are used to change the shape of an ndarray, but they work differently. reshape() returns a new array with the new shape without modifying the original data and often creates a view if possible, so the original array remains unchanged. In contrast, resize() modifies the original array directly and may add or remove elements if the new size is different from the original size. Therefore, reshape() is usually safer when you only want to change how the data is viewed, while resize() is used when you really want to change the array size.

[VN] Slicing trong NumPy khác gì với slicing của Python list?

[EN] How is slicing in NumPy different from slicing in Python lists?

[VN] Trong NumPy, slicing thường trả về một view của mảng gốc thay vì tạo bản sao dữ liệu mới. Điều này có nghĩa là nếu ta thay đổi dữ liệu trong phần slice thì mảng ban đầu cũng sẽ thay đổi. Cách này giúp NumPy tiết kiệm bộ nhớ và tăng hiệu năng. Ngược lại, khi slicing với Python list, kết quả thường là một bản sao (copy) của dữ liệu, nên nếu thay đổi list mới thì list ban đầu sẽ không bị ảnh hưởng.

[EN] In NumPy, slicing usually returns a view of the original array instead of creating a new copy of the data. This means that if you modify the sliced result, the original array will also change. This behavior helps NumPy save memory and improve performance. In contrast, slicing a Python list typically returns a copy of the data, so changes to the new list do not affect the original list.

[VN] Copy vs View trong NumPy là gì?

[EN] What are Copy and View in NumPy?

[VN] Trong NumPy, view và copy là hai cách truy cập dữ liệu của một ndarray. View là một “cái nhìn” khác của cùng vùng nhớ với mảng gốc, nghĩa là nó không tạo dữ liệu mới mà chỉ tham chiếu đến dữ liệu hiện có. Vì vậy nếu thay đổi dữ liệu trong view thì mảng gốc cũng sẽ thay đổi. Ngược lại, copy tạo ra một bản sao hoàn toàn mới của dữ liệu trong

bộ nhớ, nên khi thay đổi copy thì mảng gốc sẽ không bị ảnh hưởng. View giúp tiết kiệm bộ nhớ và nhanh hơn, còn copy an toàn hơn khi cần dữ liệu độc lập.

[EN] In NumPy, view and copy are two ways to access data in an ndarray. A view is another representation of the same memory as the original array, meaning it does not create new data but only references the existing data. Because of this, changing the view will also change the original array. In contrast, a copy creates a completely new dataset in memory, so modifying the copy does not affect the original array. A view is more memory-efficient and faster, while a copy is safer when independent data is needed.

[VN] Khi nào NumPy giúp tăng performance đáng kể?

[EN] When does NumPy significantly improve performance?

[VN] NumPy giúp tăng performance đáng kể khi làm việc với dữ liệu lớn và các phép tính số học trên mảng. Điều này xảy ra khi cần thực hiện nhiều phép toán lặp lại như cộng, nhân, thống kê hoặc xử lý ma trận. NumPy nhanh hơn vì nó sử dụng vectorized operations, dữ liệu được lưu liên tục trong bộ nhớ và các phép tính được thực hiện bằng code tối ưu viết bằng C thay vì vòng lặp Python. Vì vậy NumPy rất hiệu quả trong các tác vụ như data processing, scientific computing, machine learning và batch calculations.

[EN] NumPy significantly improves performance when working with large datasets and numerical operations on arrays. This usually happens when many repeated calculations are needed, such as addition, multiplication, statistics, or matrix operations. NumPy is faster because it uses vectorized operations, stores data in contiguous memory, and performs computations using optimized C code instead of Python loops. For this reason, NumPy is very effective in tasks like data processing, scientific computing, machine learning, and batch calculations.

[VN] NumPy lưu dữ liệu trong memory như thế nào?

[EN] How does NumPy store data in memory?

[VN] Trong NumPy, dữ liệu trong ndarray được lưu liên tục trong bộ nhớ (contiguous memory) và tất cả phần tử phải có cùng một kiểu dữ liệu (dtype). Điều này giúp NumPy truy cập dữ liệu nhanh hơn và thực hiện các phép toán hiệu quả hơn so với Python list. Ngoài ra, ndarray còn sử dụng thông tin như shape, dtype và strides để xác định cách dữ liệu được tổ chức và truy cập trong bộ nhớ. Nhờ cấu trúc này, NumPy có thể xử lý các phép tính trên mảng lớn rất nhanh và tiết kiệm bộ nhớ.

[EN] In NumPy, data in an ndarray is stored in contiguous memory, and all elements must have the same data type (dtype). This design allows NumPy to access data faster and perform computations more efficiently compared to Python lists. The ndarray also uses

metadata such as shape, dtype, and strides to describe how the data is organized and accessed in memory. Because of this structure, NumPy can process large array computations very quickly and with better memory efficiency.

[VN] Sự khác nhau giữa np.array() và np.asarray() là gì?

[EN] What is the difference between np.array() and np.asarray()?

[VN] Trong NumPy, np.array() và np.asarray() đều dùng để tạo ndarray từ dữ liệu như list hoặc tuple, nhưng cách xử lý bộ nhớ khác nhau. np.array() thường tạo một bản sao dữ liệu mới (copy), vì vậy mảng kết quả độc lập với dữ liệu ban đầu. Trong khi đó, np.asarray() sẽ không tạo copy nếu dữ liệu đầu vào đã là một ndarray, mà chỉ trả về chính mảng đó hoặc một view của nó. Vì vậy np.asarray() thường nhanh hơn và tiết kiệm bộ nhớ hơn khi dữ liệu đã ở dạng NumPy array.

[EN] In NumPy, np.array() and np.asarray() are both used to create an ndarray from data such as a list or tuple, but they handle memory differently. np.array() usually creates a new copy of the data, so the resulting array is independent from the original data. In contrast, np.asarray() does not create a copy if the input is already an ndarray, and simply returns the same array or a view of it. Because of this, np.asarray() is usually faster and more memory-efficient when the data is already a NumPy array.

[VN] Làm sao tính toán nhanh trên một mảng lớn trong NumPy?

[EN] How can you perform fast computations on a large array in NumPy?

[VN] Để tính toán nhanh trên một mảng lớn trong NumPy, nên sử dụng vectorized operations và các hàm có sẵn của NumPy thay vì viết vòng lặp Python. Ví dụ có thể dùng các hàm như np.sum(), np.mean(), np.min() hoặc thực hiện phép toán trực tiếp trên toàn bộ mảng. Những phép toán này được thực thi bằng code tối ưu viết bằng C và làm việc trên dữ liệu lưu liên tục trong bộ nhớ, nên nhanh hơn nhiều so với việc lặp qua từng phần tử bằng Python loop.

[EN] To compute quickly on a large array in NumPy, it is best to use vectorized operations and built-in NumPy functions instead of writing Python loops. For example, functions like np.sum(), np.mean(), or np.min() can operate directly on the whole array. These operations are implemented using optimized C code and work on contiguous memory, which makes them much faster than iterating through elements with Python loops.

[VN] Boolean indexing trong NumPy là gì?

[EN] What is Boolean indexing in NumPy?

[VN] Trong NumPy, Boolean indexing là kỹ thuật dùng một mảng giá trị True và False để lọc hoặc chọn các phần tử trong ndarray. NumPy sẽ kiểm tra điều kiện cho từng phần tử và

chỉ giữ lại những phần tử có giá trị True. Cách này giúp lọc dữ liệu nhanh và dễ đọc hơn so với việc dùng vòng lặp Python, và thường được dùng để tìm, lọc hoặc cập nhật các phần tử thỏa điều kiện trong mảng.

[EN] In NumPy, Boolean indexing is a technique that uses an array of True and False values to filter or select elements from an ndarray. NumPy checks the condition for each element and keeps only the elements where the condition is True. This method makes data filtering faster and more readable compared to using Python loops, and it is commonly used to find, filter, or update elements that meet specific conditions in an array.

[VN] NumPy thường được dùng chung với thư viện nào trong data pipeline?

[EN] Which libraries is NumPy commonly used with in a data pipeline?

[VN] Trong data pipeline, NumPy thường được dùng chung với nhiều thư viện khác để xử lý và phân tích dữ liệu. Ví dụ, Pandas dùng để xử lý dữ liệu dạng bảng như CSV hoặc database; SciPy dùng cho các phép tính khoa học và toán học nâng cao; còn các framework machine learning như TensorFlow và PyTorch cũng sử dụng NumPy để xử lý dữ liệu đầu vào và thực hiện các phép toán trên tensor. Nhờ đó NumPy thường đóng vai trò nền tảng cho nhiều bước trong data processing và machine learning pipeline.

[EN] In a data pipeline, NumPy is commonly used together with other libraries for data processing and analysis. For example, Pandas is used for working with tabular data such as CSV files or databases; SciPy is used for advanced scientific and mathematical computations; and machine learning frameworks like TensorFlow and PyTorch also rely on NumPy for handling input data and numerical operations on tensors. Because of this, NumPy often serves as a core foundation in many data processing and machine learning pipelines.

[VN] Khi nào không nên dùng NumPy?

[EN] When should you not use NumPy?

[VN] Không nên dùng NumPy khi dữ liệu nhỏ hoặc không cần nhiều phép tính số học, vì khi đó Python list đơn giản và dễ dùng hơn. NumPy cũng không phù hợp khi dữ liệu có nhiều kiểu dữ liệu khác nhau, vì ndarray yêu cầu tất cả phần tử phải cùng một dtype. Ngoài ra, trong các logic backend thông thường như xử lý request API, business logic hoặc thao tác với object phức tạp, NumPy thường không mang lại lợi ích về hiệu năng và có thể làm code khó đọc hơn.

[EN] It is not recommended to use NumPy when the data is small or does not require heavy numerical computation, because Python lists are simpler and easier to use in those cases. NumPy is also not suitable when the data contains multiple data types, since an

ndarray requires all elements to have the same dtype. In addition, for typical backend tasks such as handling API requests, business logic, or working with complex objects, NumPy usually does not provide performance benefits and may make the code harder to read.

[VN] Goroutine hoạt động như thế nào?

[EN] How do goroutines work?

[VN] Goroutine là một lightweight thread trong Go, được quản lý bởi Go runtime thay vì hệ điều hành. Khi tạo một goroutine bằng từ khóa `go`, nó sẽ chạy song song với các goroutine khác. Go runtime sử dụng cơ chế scheduler để phân phối nhiều goroutine lên một số ít OS thread, giúp tiết kiệm tài nguyên và hỗ trợ hàng nghìn goroutine cùng lúc. Goroutine có stack nhỏ và có thể tự động mở rộng khi cần. Việc giao tiếp giữa các goroutine thường thông qua channel để đảm bảo đồng bộ và tránh race condition.

[EN] A goroutine is a lightweight thread in Go, managed by the Go runtime instead of the operating system. When a goroutine is created using the `go` keyword, it runs concurrently with other goroutines. The Go runtime uses a scheduler to map many goroutines onto a smaller number of OS threads, which saves resources and allows thousands of goroutines to run efficiently. Goroutines start with a small stack that can grow automatically when needed. Communication between goroutines is usually done through channels to ensure synchronization and avoid race conditions.

[VN] Channel là gì? Blocking vs non-blocking?

[EN] What is a channel? Blocking vs non-blocking?

[VN] Channel trong Go là một cơ chế dùng để giao tiếp và đồng bộ giữa các goroutine. Thay vì chia sẻ bộ nhớ trực tiếp, các goroutine gửi và nhận dữ liệu thông qua channel, giúp tránh race condition và làm code an toàn hơn. Khi một goroutine gửi dữ liệu vào channel hoặc nhận dữ liệu từ channel, thao tác này mặc định là blocking, nghĩa là nó sẽ chờ cho đến khi có goroutine khác nhận hoặc gửi tương ứng. Ví dụ, gửi vào unbuffered channel sẽ chờ đến khi có người nhận. Non-blocking có thể được thực hiện bằng cách dùng `select` với default, nếu không có dữ liệu sẵn sàng thì chương trình sẽ tiếp tục chạy mà không chờ. Cách này hữu ích khi không muốn goroutine bị treo.

[EN] A channel in Go is a mechanism used for communication and synchronization between goroutines. Instead of sharing memory directly, goroutines send and receive data through channels, which helps prevent race conditions and makes the code safer. By default, sending to or receiving from a channel is blocking, meaning the operation waits until the corresponding receive or send is ready. For example, sending to an unbuffered channel blocks until another goroutine receives the value. Non-blocking behavior can be

implemented using select with a default case; if no data is ready, the program continues without waiting. This is useful when you do not want a goroutine to be blocked.

[VN] So sánh concurrency model Go vs Python.

[EN] Compare the concurrency model of Go vs Python.

[VN] Concurrency model của Go dựa trên goroutine và channel. Goroutine là lightweight thread được quản lý bởi Go runtime và có thể chạy song song thực sự trên nhiều core CPU. Go không có Global Interpreter Lock nên có thể thực hiện true parallelism cho cả CPU-bound và I/O-bound task. Channel được dùng để giao tiếp và đồng bộ giữa các goroutine theo mô hình “share memory by communicating”. Trong khi đó, Python (CPython) sử dụng thread hoặc async/await. Do có GIL, Python không thể thực hiện true parallelism với multi-threading cho CPU-bound task, nhưng vẫn hiệu quả cho I/O-bound nhờ async hoặc khi thread nhả GIL. Nếu cần xử lý song song CPU nặng trong Python, phải dùng multiprocessing.

[EN] Go’s concurrency model is based on goroutines and channels. Goroutines are lightweight threads managed by the Go runtime and can run truly in parallel on multiple CPU cores. Go does not have a Global Interpreter Lock, so it supports true parallelism for both CPU-bound and I/O-bound tasks. Channels are used for communication and synchronization between goroutines following the “share memory by communicating” principle. In contrast, Python (CPython) uses threads or async/await for concurrency. Because of the GIL, Python cannot achieve true parallelism with multi-threading for CPU-bound tasks, but it works well for I/O-bound tasks when threads release the GIL or when using async. For heavy CPU parallel processing in Python, multiprocessing is required.

[VN] Context trong Go dùng để làm gì?

[EN] What is context used for in Go?

[VN] Context trong Go được dùng để truyền thông tin điều khiển giữa các goroutine, đặc biệt trong các request dài hoặc hệ thống phân tán. Nó thường được dùng để quản lý timeout, cancel và truyền giá trị theo request như request ID hoặc user information. Khi một request bị hủy hoặc hết thời gian chờ, context sẽ gửi tín hiệu để các goroutine liên quan dừng lại, tránh rò rỉ tài nguyên. Context giúp đảm bảo hệ thống phản hồi nhanh hơn và không tiếp tục xử lý khi không còn cần thiết.

[EN] Context in Go is used to pass control information between goroutines, especially in long-running requests or distributed systems. It is commonly used to manage timeouts, cancellations, and to carry request-scoped values such as request ID or user information. When a request is canceled or times out, the context sends a signal to related goroutines to

stop processing, preventing resource leaks. Context helps ensure the system responds quickly and does not continue unnecessary work.

[VN] Memory management trong Go ra sao?

[EN] How does memory management work in Go?

[VN] Memory management trong Go được quản lý tự động bởi Garbage Collector (GC), nên lập trình viên không cần cấp phát và giải phóng bộ nhớ thủ công như trong C/C++. Khi tạo biến hoặc struct, Go sẽ tự động cấp phát trên stack hoặc heap tùy theo escape analysis. Nếu một biến chỉ dùng trong phạm vi hàm, nó thường được đặt trên stack; nếu nó cần tồn tại lâu hơn hoặc được tham chiếu bên ngoài, nó sẽ được đặt trên heap. Garbage Collector sẽ định kỳ thu gom các object không còn được tham chiếu để giải phóng bộ nhớ. Go sử dụng concurrent, low-latency GC để giảm thời gian dừng chương trình, giúp hệ thống vẫn ổn định và hiệu năng tốt.

[EN] Memory management in Go is handled automatically by a Garbage Collector (GC), so developers do not need manual memory allocation and deallocation like in C/C++. When variables or structs are created, Go decides whether to allocate them on the stack or heap based on escape analysis. If a variable is only used inside a function, it is usually placed on the stack; if it needs to live longer or be referenced outside, it is allocated on the heap. The Garbage Collector periodically frees objects that are no longer referenced. Go uses a concurrent, low-latency GC to minimize pause time, helping maintain stable performance and responsiveness.

[VN] Làm sao implement graceful shutdown trong Gin?

[EN] How to implement graceful shutdown in Gin?

[VN] Để implement graceful shutdown trong Gin, tôi không dùng `router.Run()` trực tiếp mà tạo một `http.Server` và chạy nó trong goroutine. Tôi lắng nghe tín hiệu hệ điều hành như `SIGINT` hoặc `SIGTERM` bằng `os/signal`. Khi nhận được tín hiệu dừng (ví dụ khi container bị stop), tôi gọi `server.Shutdown(context)` với timeout phù hợp. Cách này cho phép server ngừng nhận request mới nhưng vẫn hoàn thành các request đang xử lý trước khi tắt hẳn. Tôi cũng đảm bảo đóng database connection, message queue hoặc resource khác trong quá trình shutdown. Cách này giúp tránh mất request và đảm bảo hệ thống tắt an toàn trong môi trường production hoặc Kubernetes.

[EN] To implement graceful shutdown in Gin, I do not use `router.Run()` directly. Instead, I create an `http.Server` and run it in a goroutine. I listen for OS signals such as `SIGINT` or `SIGTERM` using `os/signal`. When a shutdown signal is received (for example when a container is stopped), I call `server.Shutdown(context)` with a proper timeout. This allows the server to stop accepting new requests but finish processing ongoing requests before

shutting down completely. I also make sure to close database connections, message queues, or other resources during shutdown. This approach prevents request loss and ensures safe shutdown in production or Kubernetes environments.

[VN] Race condition là gì? Cách detect?

[EN] What is a race condition? How to detect it?

[VN] Race condition là tình huống xảy ra khi nhiều goroutine hoặc thread truy cập và thay đổi cùng một dữ liệu đồng thời mà không có cơ chế đồng bộ phù hợp, dẫn đến kết quả không xác định hoặc sai lệch. Lỗi này thường khó phát hiện vì có thể không xảy ra thường xuyên và phụ thuộc vào timing của chương trình. Để detect race condition trong Go, tôi sử dụng công cụ built-in như `go run -race` hoặc `go test -race`, nó sẽ phân tích và cảnh báo khi có truy cập đồng thời không an toàn vào bộ nhớ. Ngoài ra, có thể review code để kiểm tra việc dùng shared variable và đảm bảo sử dụng mutex, channel hoặc cơ chế đồng bộ phù hợp để tránh race condition.

[EN] A race condition happens when multiple goroutines or threads access and modify the same shared data at the same time without proper synchronization, leading to unpredictable or incorrect results. This issue is often hard to detect because it may not happen consistently and depends on execution timing. To detect race conditions in Go, I use the built-in tool such as `go run -race` or `go test -race`, which analyzes the program and reports unsafe concurrent memory access. Additionally, I review code carefully to check shared variables and ensure proper use of synchronization mechanisms like mutexes, channels, or other safe concurrency patterns to prevent race conditions.

[VN] Struct embedding vs inheritance.

[EN] Struct embedding vs inheritance.

[VN] Struct embedding trong Go là cách tái sử dụng code bằng cách nhúng một struct vào struct khác. Khi embed, struct bên ngoài sẽ có quyền truy cập trực tiếp vào field và method của struct bên trong, giống như được “kế thừa” nhưng không phải inheritance thực sự. Go không hỗ trợ inheritance như trong Java hoặc OOP truyền thống. Thay vào đó, Go khuyến khích composition thay vì inheritance để giảm coupling và tăng tính linh hoạt. Inheritance cho phép một class kế thừa thuộc tính và hành vi từ class cha, nhưng có thể tạo hierarchy phức tạp và tight coupling. Struct embedding đơn giản hơn, rõ ràng hơn và phù hợp với triết lý thiết kế của Go.

[EN] Struct embedding in Go is a way to reuse code by embedding one struct inside another struct. When a struct is embedded, the outer struct can directly access the fields and methods of the inner struct, similar to inheritance but not true inheritance. Go does not support classical inheritance like in Java or traditional OOP. Instead, Go promotes

composition over inheritance to reduce coupling and increase flexibility. Inheritance allows a class to inherit properties and behavior from a parent class, but it can create complex hierarchies and tight coupling. Struct embedding is simpler, clearer, and aligns better with Go's design philosophy.

[VN] Interface trong Go hoạt động thế nào?

[EN] How do interfaces work in Go?

[VN] Interface trong Go là một tập hợp các method signature mà một type phải implement. Điểm đặc biệt là Go sử dụng implicit implementation, nghĩa là một struct không cần khai báo rõ ràng là “implements” interface; chỉ cần nó có đầy đủ method giống với interface thì tự động được coi là implement. Điều này giúp giảm coupling và tăng tính linh hoạt trong thiết kế. Interface thường được dùng để định nghĩa behavior chung và hỗ trợ dependency injection hoặc mocking khi test. Nhờ cơ chế này, Go khuyến khích thiết kế dựa trên behavior thay vì kế thừa.

[EN] An interface in Go is a collection of method signatures that a type must implement. The special feature is that Go uses implicit implementation, meaning a struct does not need to explicitly declare that it implements an interface; if it defines all the required methods, it automatically satisfies the interface. This reduces coupling and increases flexibility in design. Interfaces are commonly used to define shared behavior and support dependency injection or mocking in tests. With this mechanism, Go encourages designing based on behavior rather than inheritance.

[VN] Làm sao tối ưu performance API Go?

[EN] How to optimize API performance in Go?

[VN] Để tối ưu performance API Go, tôi bắt đầu bằng việc đo lường bằng profiling và monitoring để tìm bottleneck. Tôi tận dụng concurrency của Go bằng goroutine hợp lý nhưng tránh tạo quá nhiều gây quá tải. Tôi tối ưu database bằng cách dùng connection pool, index phù hợp và tránh query không cần thiết. Tôi giảm allocation bộ nhớ bằng cách tái sử dụng object khi có thể và tránh tạo struct hoặc slice lớn không cần thiết. Ngoài ra, tôi cấu hình timeout, keep-alive và dùng HTTP server hiệu quả. Ở mức hệ thống, tôi deploy phía sau load balancer và scale ngang khi cần. Mục tiêu là giảm latency, tăng throughput và giữ hệ thống ổn định.

[EN] To optimize Go API performance, I start by measuring with profiling and monitoring tools to identify bottlenecks. I leverage Go's concurrency using goroutines properly but avoid creating too many that may overload the system. I optimize the database by using connection pooling, proper indexing, and avoiding unnecessary queries. I reduce memory allocation by reusing objects when possible and avoiding large unnecessary structs or

slices. Additionally, I configure timeouts, keep-alive, and use the HTTP server efficiently. At the system level, I deploy behind a load balancer and scale horizontally when needed. The goal is to reduce latency, increase throughput, and keep the system stable.

II. Programming Languages & Frameworks (Python/FastAPI + Golang)

FastAPI hoạt động dựa trên ASGI như thế nào?

async / await trong Python hoạt động ra sao?

Khi nào nên dùng async ? Khi nào không?

So sánh FastAPI và Flask về performance và concurrency.

Dependency Injection trong FastAPI là gì?

Pydantic validation hoạt động như thế nào?

Middleware trong FastAPI dùng để làm gì?

Làm sao tối ưu performance API Python?

GIL là gì? Nó ảnh hưởng thế nào tới multi-threading?

multiprocessing vs threading trong Python.

Cách implement background tasks trong FastAPI.

Rate limiting trong API nên làm ở đâu?

Làm sao handle global exception trong FastAPI?

Làm sao viết API versioning?

Bạn sẽ test API như thế nào(unit/integration)?

Python memory management hoạt động thế nào?

Reference counting và garbage collector khác nhau ra sao?

Mutable vs immutable ảnh hưởng thế nào đến performance và bug?

Deep copy vs shallow copy khác nhau thế nào?

Khi nào gây bug khó phát hiện?

Decorator hoạt động ra sao?

Ứng dụng thực tế trong logging, auth, caching?

Context manager là gì?

Khi nào nên tự viết context manager?

Metaclass là gì?

Khi nào thực sự cần dùng Metaclass?

Monkey patching là gì?

Khi nào nên và không nên dùng Monkey patching?

Python import system hoạt động thế nào?

Circular import xử lý ra sao?

Typing(type hints) giúp gì trong dự án lớn?

Khi nào nên dùng mypy?

Dataclass vs Pydantic vs NamedTuple khác nhau thế nào?

Concurrency trong Python gồm những mô hình nào?
Async IO vs threading vs multiprocessing khác nhau ra sao trong thực tế?
Event loop hoạt động như thế nào?
Blocking call ảnh hưởng gì đến async app?
Làm sao detect memory leak trong Python?
Profiling Python code bằng cProfile hoặc line_profiler như thế nào?
Time complexity ảnh hưởng thế nào đến API performance?
Thiết kế project structure cho ứng dụng lớn như thế nào?
Làm sao tách business logic khỏi framework(clean architecture)?
Dependency injection trong Python thuần(không dùng FastAPI) làm thế nào?
Khi nào nên dùng class, khi nào nên dùng function thuần?
Làm sao viết code Python thread-safe?
Gunicorn vs Uvicorn khác nhau thế nào?
Worker type ảnh hưởng ra sao?
WSGI vs ASGI khác nhau thế nào?
Làm sao scale Python app khi GIL tồn tại?
Khi nào cần dùng C extension hoặc Cython?
Mock vs patch khác nhau thế nào?
Test async function như thế nào?
Property-based testing là gì?
Coverage cao có đảm bảo code tốt không?
Closure là gì và được dùng để làm gì?
NumPy là gì? Tại sao nên dùng NumPy thay vì Python list?
ndarray trong NumPy là gì? Nó khác gì với list trong Python?
Vectorization trong NumPy là gì? Vì sao nó nhanh hơn loop Python?
Broadcasting trong NumPy là gì?
Shape và dtype của một array là gì?
Sự khác nhau giữa reshape() và resize() trong NumPy?
Slicing trong NumPy khác gì với slicing của Python list?
Copy vs View trong NumPy là gì?
Khi nào NumPy giúp tăng performance đáng kể?
NumPy lưu dữ liệu trong memory như thế nào?
Sự khác nhau giữa np.array() và np.asarray() là gì?
Làm sao tính toán nhanh trên một mảng lớn trong NumPy?
Boolean indexing trong NumPy là gì?
NumPy thường được dùng chung với thư viện nào trong data pipeline?
Khi nào không nên dùng NumPy?
Goroutine hoạt động như thế nào? **Golang**
Channel là gì? Blocking vs non-blocking?

So sánh concurrency model Go vs Python.

Context trong Go dùng để làm gì?

Memory management trong Go ra sao?

Làm sao implement graceful shutdown trong Gin?

Race condition là gì? Cách detect?

Struct embedding vs inheritance.

Interface trong Go hoạt động thế nào?

Làm sao tối ưu performance API Go?